Nodebase — From “Sign Up” to “Sign In”

Full request flow: which frontend file calls which backend file, request to response, and the rows written at each step

Next.js 15 (App Router) • Better Auth + Prisma adapter • Postgres • BullMQ 6 + Upstash Redis • Nodemailer SMTP • Google / GitHub OAuth

Path A · Email + Password · Sign Up | Path B · Email + Password · Sign In | Path C · Google / GitHub

0. Map of the journey

Four flows, three endpoints, one destination — `/workflows`. Every auth call is served by the same catch-all route and the same Better Auth instance. Sign up and sign in share the OAuth button; the outcome depends only on which endpoint fires and which rows are written.

#	UI (route)	User action	Network request	What the backend does
A	`/register` `register-form.tsx`	Fill name/email/password, click **Create account**	POST `/api/auth/sign-up/email`	Create `User` + credential `Account`, write an OTP `Verification` row, enqueue an OTP email on BullMQ. **No session** → redirect to `/verify-email`.
B	`/login` `login-form.tsx`	Fill email/password, click **Sign in**	POST `/api/auth/sign-in/email`	If verified: create a `Session` + cookie → `/workflows`. If not verified: throw **403**, email an OTP (`type:"sign-in"`), redirect to `/verify-email`.
C	`/register` or `/login`	Click **Continue with Google / GitHub**	GET `/api/auth/sign-in/social` → provider → GET `/api/auth/callback/:provider`	Full-page redirect out. On callback: upsert `Account`, create or link `User`, create `Session` + cookie → `/workflows`. **No OTP.**

Every file in the chain (frontend)

File	Role in this flow
`src/app/(auth)/register/page.tsx`	Server component; `requireUnauth()` then renders `<RegisterForm/>`
`src/app/(auth)/login/page.tsx`	Server component; `requireUnauth()` then renders `<LoginForm/>`
`src/app/(auth)/verify-email/page.tsx`	Hosts the OTP entry step that completes sign-up / unverified login
`src/features/auth/components/register-form.tsx`	Sign-up form + OAuth buttons; zod schema at `:31`, `signUp.email` at `:89`, redirect at `:98`
`src/features/auth/components/login-form.tsx`	Sign-in form + OAuth buttons; lax schema at `:31`, `signIn.email` at `:49`, the `403` branch at `:57`
`src/features/auth/components/verify-email-form.tsx`	OTP entry; calls `emailOtp.verifyEmail` → session created on success
`src/lib/auth-client.ts`	Browser auth client registering `polarClient()` + `emailOTPClient()`, so calls are plugin-aware
`src/lib/auth-utils.ts`	`requireAuth()` / `requireUnauth()` server guards

Every file in the chain (backend & infrastructure)

File	Role in this flow
`src/app/api/auth/[...all]/route.ts:4`	`export const { POST, GET } = toNextJsHandler(auth)` — serves every auth endpoint
`src/lib/auth.ts`	The configuration: `emailAndPassword` (:53), `emailVerification` (:58), `socialProviders` (:63), `trustedOrigins` (:74), `rateLimit` (:83), `emailOTP` (:112)
`src/lib/otp-limit.ts`	60 s cooldown + hourly OTP budget via Upstash rate limiter

`src/lib/queue/email-queue.ts`	Lazily-created BullMQ `email` queue; `enqueueOtpEmail()`
`src/queue/worker.ts`	Separate process, concurrency 2, drains the `email` queue and calls `sendOtpEmail`
`src/lib/email.ts`	Nodemailer transport + the OTP email template
`prisma/schema.prisma`	`User` (:17), `Session` (:34), `Account` (:49), `Verification` (:68)

1. Route & file tree (what the clicks render)

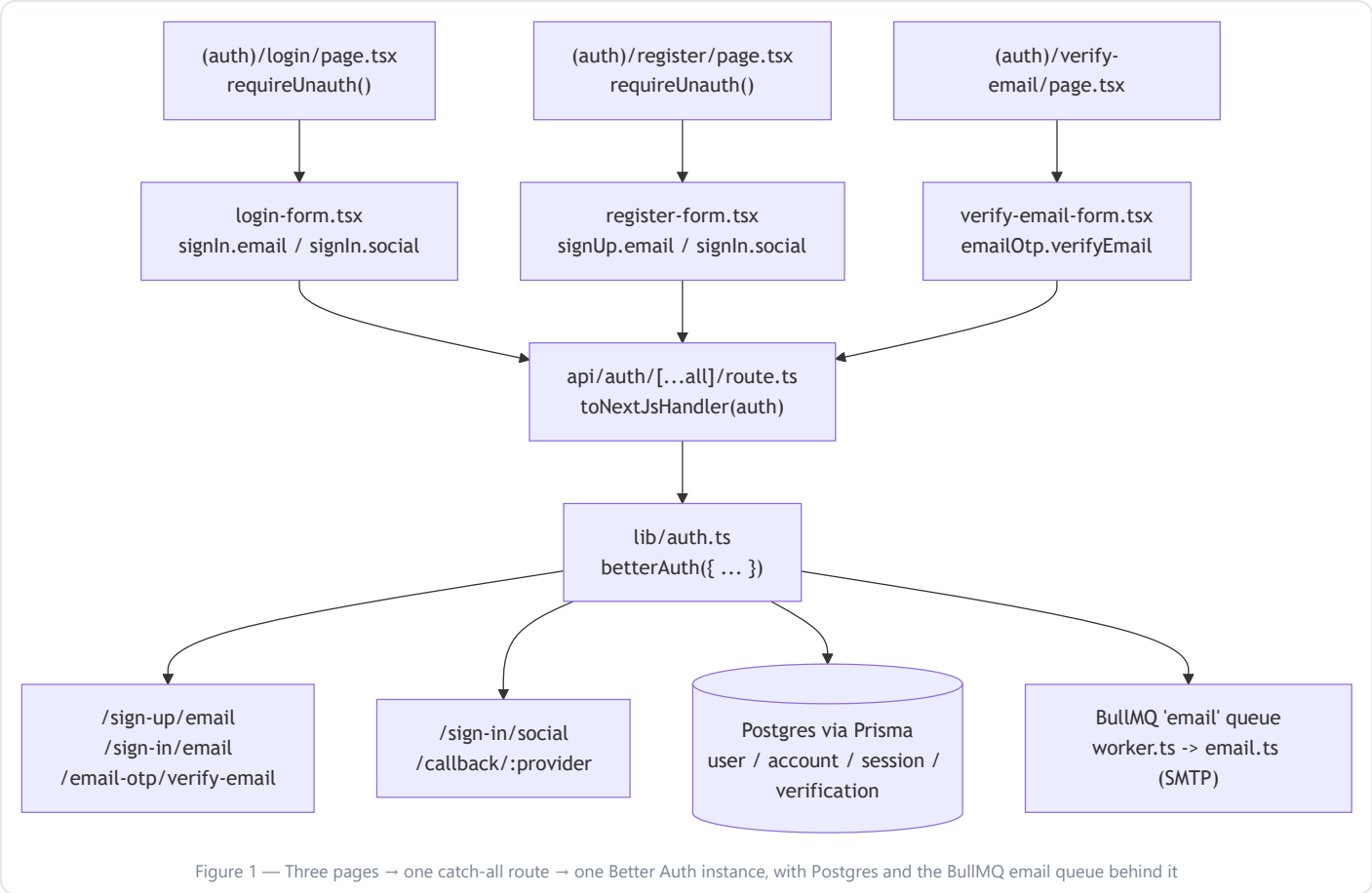


Figure 1 — Three pages → one catch-all route → one Better Auth instance, with Postgres and the BullMQ email queue behind it

Both pages call `requireUnauth()` so an already-authenticated user is bounced straight to `/workflows` and never sees the form. Every form action funnels through the single catch-all handler into the shared `auth` object.

> **Ambiguity to watch:** the client branches on the HTTP status code alone. Both an unverified email and a rejected origin return `403`, so a misconfigured `trustedOrigins` looks exactly like “email not verified”. See §4.

2. Path A — Email + password sign up (A Create account)

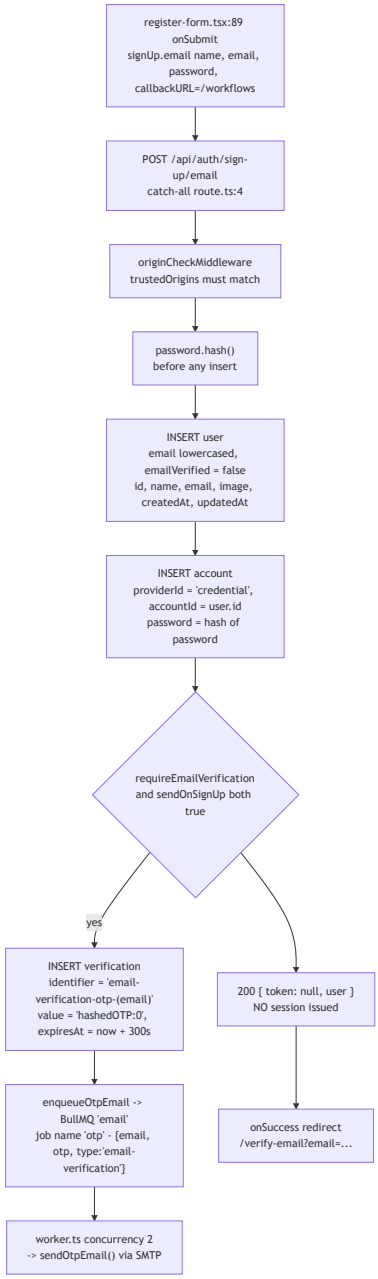


Figure 2 — signUp.email: the full request/response chain, file by file. Three rows and one queue job, returning without a session

2.1 Request (what crosses the wire)

Property	Value	Where
Method + path	POST `/api/auth/sign-up/email`	`route.ts:4` (catch-all)
Body	`{ name, email, password, callbackURL: "/workflows" }`	`register-form.tsx:89`
Client validation	`name` min(1), `email` valid, `password` min(8), confirm must match	`register-form.tsx:31-41`
Origin	Must match `trustedOrigins` (`NEXT_PUBLIC_APP_URL` , `BETTER_AUTH_URL` , localhost)	`auth.ts:74-81`

2.2 Backend processing, in order

1. **Origin check** — `originCheckMiddleware` rejects a mismatched origin with `403 Invalid origin` *before* anything else runs.
2. **Rate limit** — global window 20/60 s; the OTP endpoint is capped at 3/60 s and verify at 8/300 s (`auth.ts:83-91`).
3. **Hash the password** — done before any insert, so a hashing failure cannot leave a half-created account.

4. **INSERT `user`** — `email` lower-cased, `name`, `image` null (this form does not send one), `emailVerified: false`, plus generated `id` and timestamps. A duplicate email trips the unique index → `UNPROCESSABLE_ENTITY / FAILED_TO_CREATE_USER`.
5. **INSERT `account`** via `linkAccount` — `providerId: "credential"`, `accountId = user.id`, `userId`, `password = <hash>`; every OAuth token field stays null.
6. **Verification + OTP** — because `requireEmailVerification` (`auth.ts:56`) and `sendOnSignUp` (`auth.ts:60`) are both set, Better Auth calls the sender. `overrideDefaultEmailVerification: true` (`auth.ts:117`) reroutes it into the OTP plugin endpoint, which writes the `verification` row and invokes `sendVerificationOTP`.
7. **Queue the email** — `sendVerificationOTP` → `enqueueOtpEmail({ email, otp, type: "email-verification" })` (`auth.ts:118`) → job `{ name:"otp", data }` on the BullMQ `email` queue.
8. **Return without a session** — `requireEmailVerification` short-circuits before `createSession`, so the response is `{ token: null, user }`.

2.3 Database rows written

Table	Fields	Values
`user`	`id, name, email, image, emailVerified, createdAt, updatedAt`	email lower-cased; `emailVerified=false`; image null
`account`	`id, accountId, providerId, userId, password, createdAt, updatedAt`	`providerId="credential"`, `accountId=userId`, hashed password
`verification`	`id, identifier, value, expiresAt, createdAt, updatedAt`	`identifier="email-verification-otp-<email>"`, `value="<hash>:0"`, +300 s
`session`	—	*none* at this stage

> **Note:** `verification.value` is `<hashedOTP>:<attempts>`. The `:0` is the failed-attempt counter (max `allowedAttempts: 5`). The plaintext OTP never touches Postgres — only the BullMQ job payload carries it.

3. Finishing sign up — the OTP step (A / B)

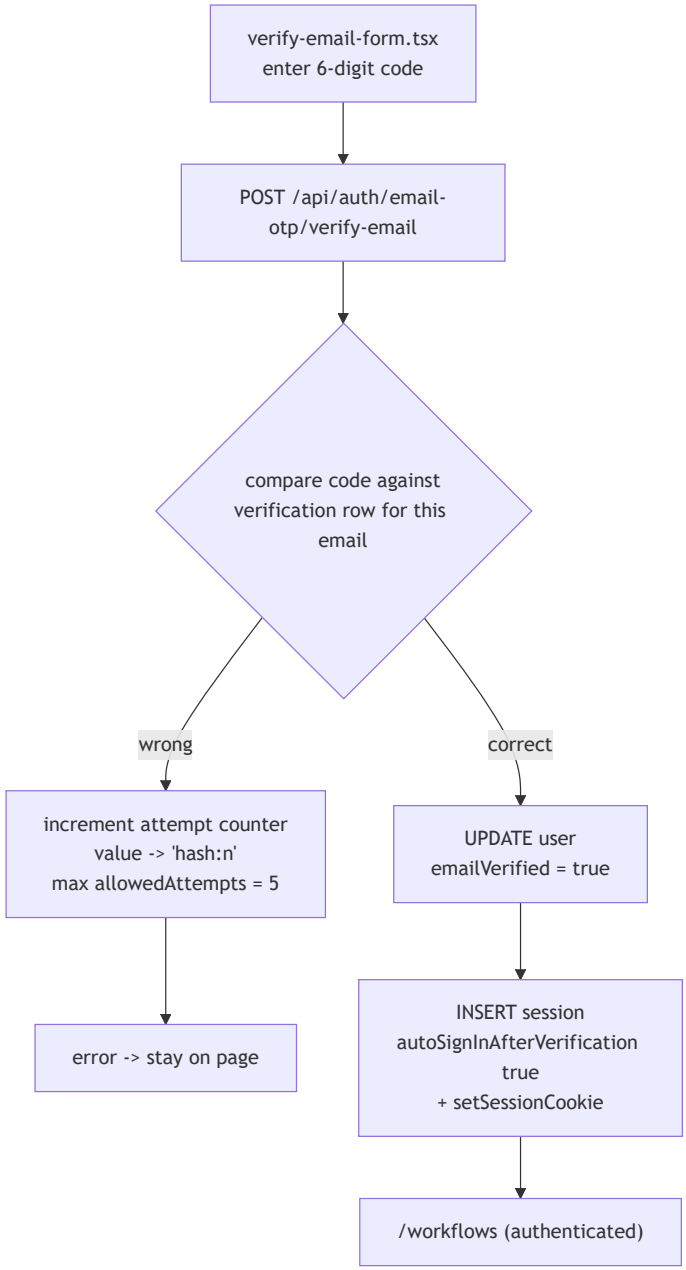


Figure 3 — Entering the code marks the user verified and signs them in, in one request

Stage	Detail
Form redirect	Sign-up `onSuccess` pushes to `/verify-email?email=<address>` (`register-form.tsx:98`)
Verify request	POST `/api/auth/email-otp/verify-email` with the 6-digit code
Check	Compare against the `verification` row for `email-verification-otp-<email>`; a wrong code increments the `:n` counter
On success	UPDATE `user.emailVerified = true`, then INSERT a `session` because `autoSignInAfterVerification: true` (`auth.ts:59`); cookie set → `/workflows`
On failure	Error surfaced; user stays on the page (max `allowedAttempts: 5`)

Coupling worth knowing: if `overrideDefaultEmailVerification` were ever turned off, the `if (!sendVerificationEmail)` guard would short-circuit — the user would hit the `403`, be redirected to `/verify-email`, and no email would ever be sent. The two settings must change together.

4. Path B — Email + password sign in (B Sign in)

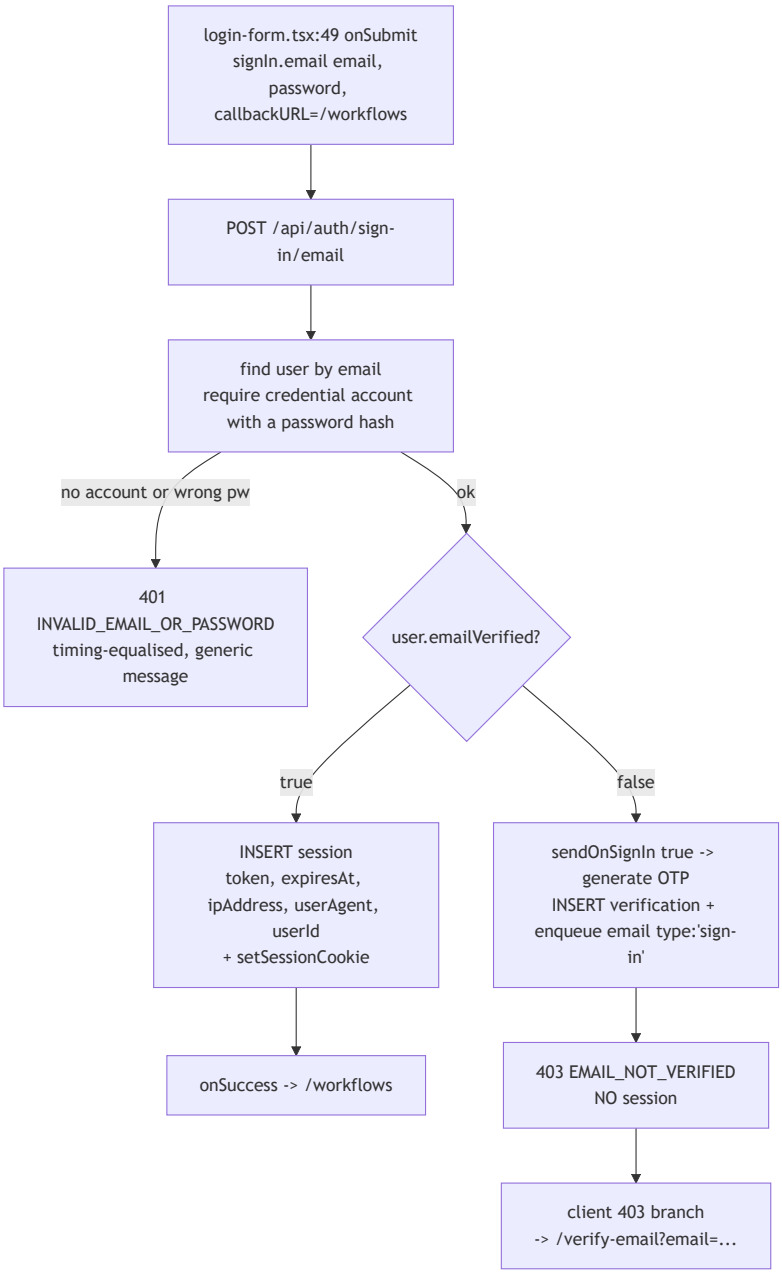


Figure 4 — signIn.email: two branches from one endpoint — a session, or a 403 that re-uses the whole OTP machinery

4.1 Request

Property	Value	Where
Method + path	POST `/api/auth/sign-in/email`	`route.ts:4`
Body	`{ email, password, callbackURL: "/workflows" }`	`login-form.tsx:49-53`
Client validation	`email` valid, `password` min(1) only	`login-form.tsx:31`

Login's schema is deliberately laxer than sign-up's: length rules belong to account creation, and applying `min(8)` at login would reject legacy accounts. A wrong password cannot be caught client-side, so it round-trips to the server.

4.2 Backend processing, in order

1. **Origin check** first, as always.
2. **Find user** with accounts attached, then require a `credential` account that actually holds a `password`. On any miss it hashes the supplied password anyway (timing equalisation) and throws `401 INVALID_EMAIL_OR_PASSWORD` — a generic message that never discloses whether the

account exists or is OAuth-only.

3. **Password hash compared.** Mismatch → the same generic `401`.
4. **Evaluate** `requireEmailVerification && !emailVerified`.
   - **false** → `createSession()` writes a `session` row (`token, expiresAt, ipAddress, userAgent, userId`) and `setSessionCookie` signs the token into the browser.
   - **true** → the 403 branch below.
5. **Response.** A failed `createSession` returns `401 FAILED_TO_CREATE_SESSION` — the handler never reports success without a durable session.

4.3 The unverified 403 branch

When the password is right but `emailVerified === false`, Better Auth checks the sender is registered, and because `sendOnSignIn: true` (`auth.ts:61`) writes a fresh `verification` row and enqueues the OTP with `type: "sign-in"` — the exact same BullMQ path as sign-up — then throws `403 EMAIL_NOT_VERIFIED`. No session is created.

The client (`login-form.tsx:61`) treats the `403` as "needs verification" and redirects to `/verify-email?email=...`, where `autoSignInAfterVerification` completes the login when the code is entered.

> **Two errors, one status code:** `EMAIL_NOT_VERIFIED` (from the sign-in handler) and `Invalid origin` (from `originCheckMiddleware`) both return `403`. The client cannot tell them apart, so a bad `trustedOrigins` silently redirects the user to `/verify-email` with no toast. Branching on `ctx.error.code === "EMAIL_NOT_VERIFIED"` instead of the status would fix it.

4.4 Session created on success

Table	Fields	Values
`session`	`id, token, expiresAt, ipAddress, userAgent, userId, createdAt, updatedAt`	Fresh row per login; cookie carries the signed token

5. Path C — Google / GitHub sign up or sign in (C)

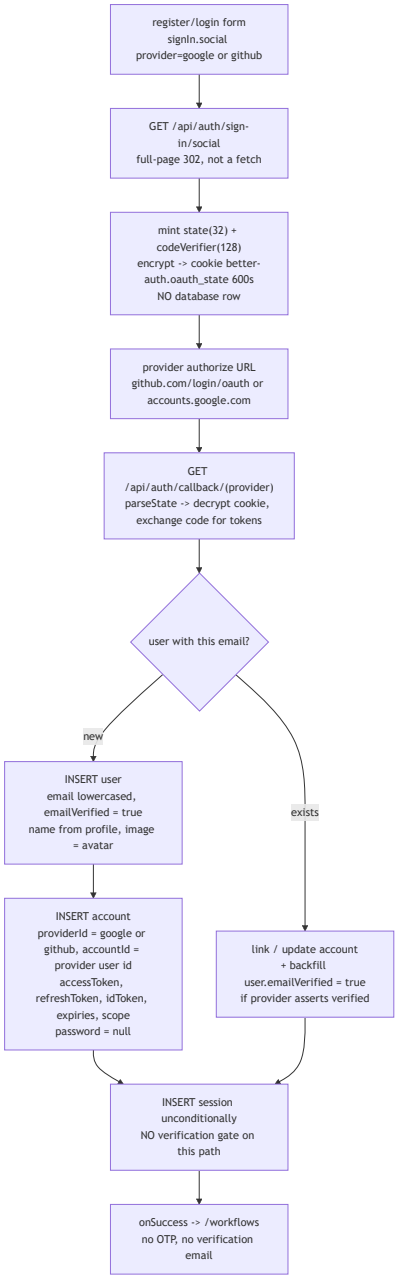


Figure 5 — Out to the provider and back to the callback: a brand-new user creates a user, an account, and a session with no OTP

5.1 Authorize redirect

Property	Value	Where
Trigger	`authClient.signIn.social({ provider })` — no `callbackURL`	`register-form.tsx:59,73`, `login-form.tsx`
Redirect	`302` to the provider authorize URL with `?state=`	Better Auth
State store	Encrypted blob in cookie `better-auth.oauth_state` — **no database row**	default strategy `"cookie"`
PKCE verifier	Generated (128 chars) but **never sent** — providers declared without `pkce: true`	`auth.ts:63-71`

> **Callback registration:** because these are absolute provider redirects, the OAuth apps must be registered with `<app-url>/api/auth/callback/github` and `…/google`. Under an ngrok tunnel the origin changes on every rotation, so social sign-in breaks while email/password login keeps working.

5.2 Callback processing, in order

1. **parseState** — decrypts the `oauth_state` cookie, clears it, and enforces the 10-minute `expiresAt`.
2. **Exchange** the authorization code for tokens.
3. **No existing user** → `createOAuthUser` inserts the `user` and `account` rows (Figure 5).
   - `user.emailVerified = true` — Google and GitHub both assert `email_verified`.
   - `account.providerId` is `"google"` or `"github"`; `accountId` is the provider's user id.
4. **No OTP** — the sign-up OTP only fires when `!userInfo.emailVerified`; here it is true, so the branch is skipped.
5. **INSERT session** unconditionally — there is no `requireEmailVerification` gate anywhere in the OAuth callback — then set the cookie.
6. **Client** — `onSuccess` runs `router.push("/workflows")`.

5.3 When the email already exists (sign in via OAuth)

If a `user` with the same email already exists, Better Auth links a new `account` (or refreshes tokens on an existing one) instead of creating a user. Critically, if the provider asserts the address is verified and the stored user is not, it **back-fills** `emailVerified = true`.

> *Completing a Google sign-in with the same address clears the OTP gate on a pre-existing password account — the user can then sign in with the password without ever entering an OTP.*

Situation	user row	account row	session
New OAuth email	INSERT (verified)	INSERT (provider, tokens)	INSERT
Email exists as password account	UPDATE `emailVerified=true`	INSERT / UPDATE provider account	INSERT
Email exists via another provider	unchanged	INSERT / UPDATE this provider	INSERT

Password sign-in against an OAuth-only address is rejected cleanly — no takeover, no silent merge. `signInEmail` finds the user but no `credential` account, so it throws the generic `INVALID_EMAIL_OR_PASSWORD`; sign-up with the same address also fails on the unique index `user_email_key`. An OAuth-only user is locked to OAuth unless `/set-password` is implemented (it is not, anywhere in `src/`).

6. All three paths at a glance (sequence)

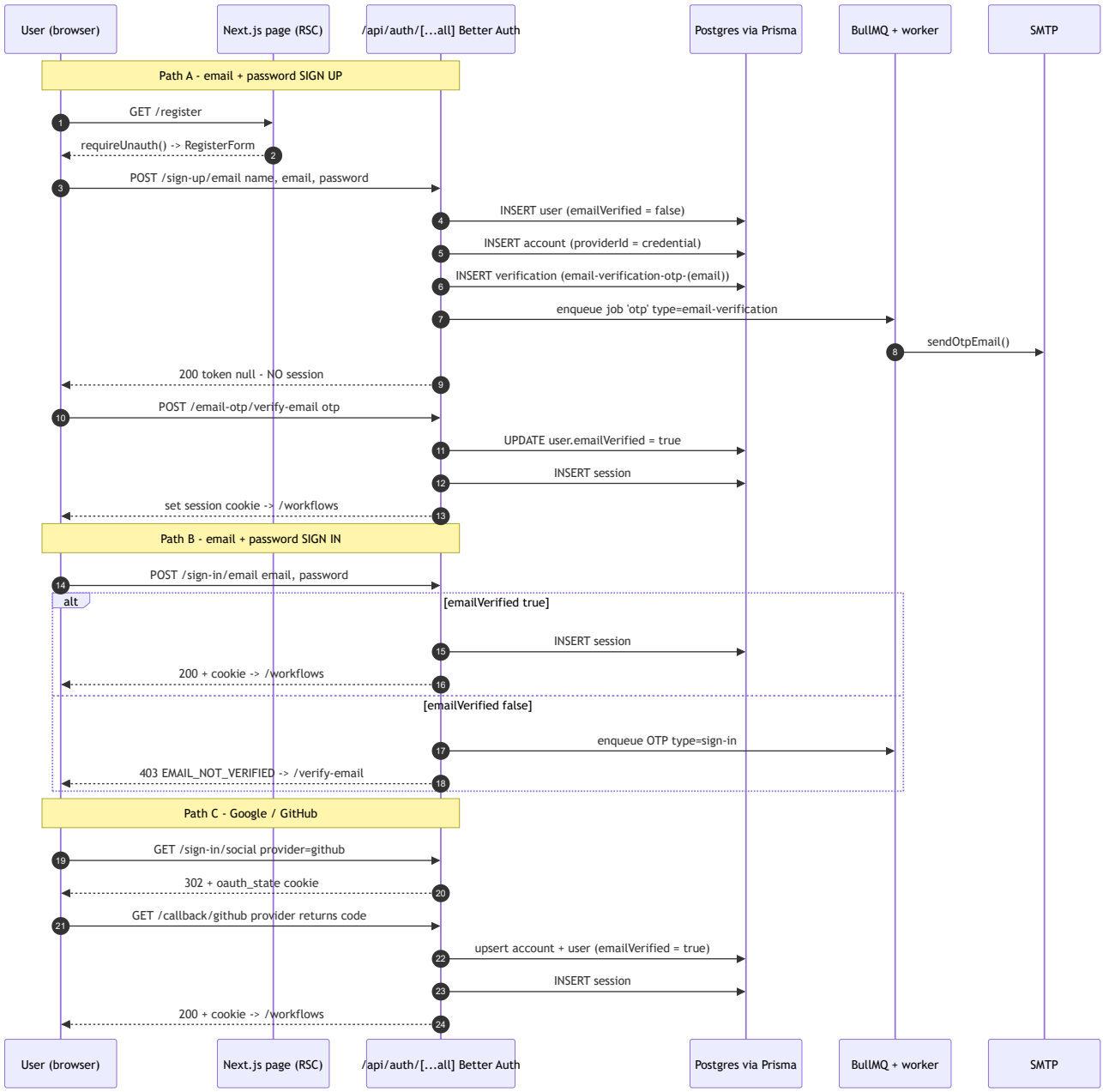


Figure 6 — The same participants for every path: the browser, the RSC page guard, the catch-all auth route, Postgres, and the BullMQ / SMTP worker

7. Data model written across the whole flow

Four auth tables back all four flows.

Table	Columns	Written by
`user`	`id, name, email, emailVerified, image, createdAt, updatedAt`	password sign-up (verified=false), OAuth (verified=true), verify step / OAuth back-fill (UPDATE)
`account`	`id, accountId, providerId, userId, password, accessToken, refreshToken, idToken, accessTokenExpiresAt, refreshTokenExpiresAt, scope, createdAt, updatedAt`	sign-up (`credential`), OAuth callback (`google` / `github`)
`session`	`id, token, expiresAt, ipAddress, userAgent, userId, createdAt, updatedAt`	password sign-in, verify step, OAuth callback
`verification`	`id, identifier, value, expiresAt, createdAt, updatedAt`	password sign-up and the unverified sign-in branch (identifier `<type>-otp-<email>`)

Side stores (Redis / SMTP), not Postgres

Store	Key / job	Written when
BullMQ queue `email`	job `otp` → `{ email, otp, type }`	every password sign-up; unverified sign-in
Upstash (idempotency)	`otp:cooldown:<email>` (SET NX EX 60)	gate before an OTP is generated
Upstash (rate limit)	`ratelimit:otp:<email>` sliding window	same gate (default 5/hour)
Nodemailer SMTP	—	worker drains the queue → `sendOtpEmail`

> **Fails open:** `enqueueOtpEmail` sends inline when `UPSTASH_REDIS_URL` is unset — no job, no retry, and no log that the queue was bypassed. The cron worker treats a missing URL as fatal; the email path does not.

8. End-to-end summary (single breath)

> ***Sign up, email + password.*** *The register form posts `{name,email,password}` to `/sign-up/email`. Origin and rate checks pass, the password is hashed, and Better Auth writes `user` (`emailVerified=false`) and a `credential` `account`. Because verification is required, the emailOTP plugin writes a `verification` row and hands the code to `enqueueOtpEmail`, which drops a job on the BullMQ `email` queue; a separate worker mails it. No session is issued, and the form redirects to `/verify-email`, where entering the code marks the user verified and creates the first `session`.*
>
> ***Sign in, email + password.*** *The login form posts to `/sign-in/email`. A verified account gets a `session` row and a signed cookie and lands on `/workflows`. An unverified account instead triggers the same OTP machinery (`type:"sign-in"`) and a `403 EMAIL_NOT_VERIFIED`, and the client redirects to `/verify-email` where verification signs the user in.*
>
> ***Sign up or sign in, Google / GitHub.*** *The button issues a full-page `302` to the provider, carrying an encrypted `oauth_state` cookie (no DB row; the PKCE verifier is generated but unused). On the callback, an unknown email creates a `user` (already verified) + `account` (provider tokens); a known email links an `account` and may back-fill `emailVerified`. A `session` is created unconditionally — no OTP, no verification gate — and the browser lands on `/workflows`.*

Line references are exact to the working tree (`src/` , `prisma/` , `src/queue/`), verified against the installed Better Auth package. Diagrams generated from the same source used to trace the flow (`.mmd` → SVG).